SEU-Resilient Network Architecture: A Novel Formal-Methods Framework for Reliable 5G/6G Systems

1st Ganesha
Network Team
Samsung R&D Institute India - Bangalore
Bangalore, India
0000-0001-9461-9231

2nd Sujit Kumar Chakrabarti
Department of Computer Science and Engineering
International Institute of Information Technology, Bangalore
Bangalore, India
0000-0001-8422-2900

2026 National Conference on Communications (NCC) | 979-8-3315-4715-8/26/$31.00 ©2026 IEEE | DOI: 10.1109/NCC68160.2026.11478915

***Abstract*—We present a novel formal-methods framework for achieving SEU-resilience in 5G/6G and emerging Non-Terrestrial Network (NTN) systems. Single-event upsets (SEUs) are increasingly observed in telecom platforms built on commercial off-the-shelf hardware, including satellite-assisted and NTN deployments where radiation exposure is higher. These faults are silent: the network continues to operate, but internal state becomes corrupted, leading to incorrect control decisions while all 3GPP signalling remains valid. Our key novelty is a property-directed analysis that identifies a small set of *conditionally relevant variables* (CRVs)—the only variables whose corruption can change the truth of a safety property. Instead of duplicating full control paths or hardening all memory, the method safeguards only CRVs, offering a low-overhead and practical route to SEU resilience. Experiments on representative OAI-derived programs show that CRV-guided protection reduces the number of variables requiring hardening by 25–40% while preventing all SEU-induced property violations. Importantly, the architecture integrates cleanly into existing Radio Access Network (RAN)/Core software and requires no modification to any 3GPP procedures, making it directly deployable in current 5G systems and naturally extendable to 6G and NTN environments.**

***Index Terms*—single-event upset (SEU), conditional relevance variables (CRVs), selective hardening, 5G/6G network control, Non-Terrestrial Networks (NTN), satellite-assisted networks, telecom software reliability, formal methods**

I. Introduction

Modern 5G and upcoming 6G networks rely on internal state values maintained across the RAN and Core. These values drive mobility handling, handover decisions, UPF selection, QoS classification, slice behaviour, etc. If any such variable is silently corrupted, the network may still appear correct at the protocol level, yet the control logic may take an incorrect decision. Prior reports show that single–event upsets (SEUs) have occurred in routers, switches, timing modules and COTS servers used in telecom deployments [1]–[5]. These faults are silent: hardware continues to operate, but stored state changes unpredictably, making SEU-induced failures difficult to detect.

An SEU is simply a transient bit-flip in memory or registers. Hardware-level defences such as ECC, parity and redundancy help reduce errors, but they add cost and still cannot prevent silent corruption of software variables. The risk is higher in satellite-connected and remote gNB deployments, where radiation exposure is greater.

Conventional safeguards such as signalling validation, integrity protection and cryptographic checks do not prevent SEU-induced failures because the corruption happens entirely inside the software state. A single bit-flip in a mobility flag, handover trigger, policy index or slice identifier can alter control behaviour while all signalling exchanges remain valid. Fully hardening all memory or duplicating complete control paths is possible, but impractical in commercial deployments due to cost, power and engineering overhead.

To address this challenge, we build on the idea of *conditionally relevant variables* (CRVs) introduced in prior work [6], [7]. A variable is conditionally relevant if changing only that variable can flip the truth of a safety property. Such variables directly influence correctness under SEU faults, whereas others do not. Identifying and protecting only this small set avoids full-system hardening.

In this paper, we extend CRV-based reasoning to key 5G/6G control functions, including gNB control loops, mobility management, handover logic, UPF selection and QoS/slice decisions. These are representative cases; the same method applies to any network routine where a bit-flip can affect the truth of a safety property. The approach integrates into existing software without modifying 3GPP procedures. By focusing protection on the variables that truly matter, the method provides a practical and low-overhead path to SEU resilience in current 5G and future 6G systems.

A. Contributions

Compared with earlier CRV work on embedded controllers [6], [7], this paper makes three contributions:

- **Adapting CRVs to 5G/6G network control.** We apply CRV–based reasoning to representative 5G/6G control logic across the RAN and Core, including mobility handling, handover decisions, QoS and slice–aware forwarding. To the best of our knowledge, this is the first use of CRVs in the context of telecom control software.

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on October 30,2026 at 08:38:53 UTC from IEEE Xplore. Restrictions apply.

- **A concrete SEU–injection and verification workflow for C network code.** We describe a fault model, instrumentation scheme and CBMC–based model–checking flow for C implementations of network control routines, so that each candidate variable can be tested directly against a given safety property.
- **Evidence that only a few variables matter for safety in 5G–style control paths.** Guided by an inspection of the OpenAirInterface (OAI) codebase, we construct four small programs that capture typical 5G decision logic and evaluate CRV analysis on them. In these examples, only a small subset of the sliced variables turn out to be conditionally relevant, supporting selective protection instead of full–state hardening.

II. Background and State of the Art

A. Background on SEUs and CRVs

A single–event upset (SEU) is a transient, radiation–induced bit flip in memory or registers [3], [4]. SEUs do not damage hardware but can silently corrupt stored state. In telecom software, such silent corruption can lead to unsafe behaviour. SEU-related faults have been reported in routers, switches, timing modules and COTS servers used in carrier networks [1], [2], [5].

Prior work introduced the notion of *conditionally relevant variables* (CRVs) for control programs [6], [7]. A variable is conditionally relevant if changing only it can alter the truth of a safety property. Variables that never affect the property are not safety-critical, enabling selective safeguarding instead of full-state hardening.

Many key 5G/6G decisions rely on stored internal values—mobility state, handover thresholds, QoS levels, slice identifiers, UPF selection indices and buffer status. If any such variable is a CRV and an SEU silently alters its value, the network may take an incorrect decision even when signalling appears valid. Identifying CRVs inside network control logic is therefore essential for reliable behaviour in both current 5G deployments and emerging 6G systems.

Conventional program-analysis techniques—such as control-flow analysis and static slicing—identify variables that syntactically influence a computation. However, they operate in a fault-free setting and therefore over-approximate relevance. A variable may appear in a slice but never be capable of violating the safety property under any single-bit flip. CRV analysis addresses this gap by focusing on *property-changing* variables rather than purely syntactic dependencies.

B. State of the Art: SEU Events in Telecom and Network Systems

Reliability studies from JEDEC and large-scale datacenter measurements show that terrestrial cosmic rays can flip bits in DRAM, caches and control logic even at ground level [3], [4]. Modern 5G Core functions run on the same COTS hardware, so a single corrupted value can influence routing, session management, RRC state or service-class decisions.

SEU behaviour has also been observed in carrier-grade routers. Cisco documented a forwarding malfunction in the ASR series that was "plausibly triggered by cosmic radiation" [1]. Juniper devices include Soft Error Recovery features in the Packet Forwarding Engine (PFE) to repair corrupted entries [2]. Such corruption may cause packet drops, bearer imbalance or incorrect scheduler decisions.

RAN equipment is similarly exposed. FPGA vendors such as AMD/Xilinx and Lattice publish SEU characterisation data and offer soft-error-mitigation IP cores for UltraScale, Versal and Nexus platforms [8]–[10]. These devices appear in DUs, radio accelerators, timing subsystems and fronthaul interfaces. ITU-T guidelines also recognise soft errors and recommend mitigation practices for telecom environments [11]. In practice, PHY, MAC and scheduler blocks implemented on FPGA logic can experience transient corruption even with ECC and regular scrubbing.

SEUs also arise in timing and satellite-backhaul components that support remote gNB deployments. Precision Time Protocol (PTP) grandmasters, synchronous Ethernet clocks and PLL-based timing circuits have released SEU test results [5]. Space-weather studies likewise report bit flips in satellite links and on-board memory [12], [13]. Any disturbance in timing or backhaul synchronisation directly affects RAN functions such as scheduling, measurement alignment and handover decisions.

Together, these observations show that SEUs are a practical and well-documented fault model across RAN, transport and Core systems. This motivates the need for software-level analysis that identifies exactly those variables whose corruption can affect the safety of 5G/6G control behaviour.

III. Identifying CRVs in Network Control: A Motivating Example

To illustrate how conditionally relevant variables (CRVs) arise in practical network software, we use a simplified but representative handover control loop from the gNB. The structure mirrors the behaviour of RRC-level mobility handling in real RAN systems. The code is intentionally compact so that the reasoning about correctness and safety remains clear. This example serves as the motivating case throughout the paper.

A. Program Behaviour

This structure captures simplified RRC mobility handling in a gNB. The loop tracks sustained poor signal conditions and allows handover only when the selected target cell matches the currently estimated best cell. The variable `ho_decision` represents the safety-critical output, while variables such as `debug_counter` do not influence correctness.

B. Safety Property

A fundamental correctness guarantee in handover control is that the UE must never be handed over to a non-best cell. This requirement is captured as the safety property ϕ:

$$\phi : \quad \neg(\texttt{ho_decision} \wedge (target_cell \neq best_cell)).$$

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on October 30,2026 at 08:38:53 UTC from IEEE Xplore. Restrictions apply.

```c
// Motivating network example for CRV analysis
void ho_loop(int ho_thresh, int bad_rsrp, int N) {

    static int rrc_state       = RRC_CONNECTED;
    static int poor_link_count = 0;   // link-quality
        tracker
    static int mobility_flag   = 1;   // UE mobility
        condition
    static int ho_decision     = 0;   // safety-critical
        output
    static int debug_counter   = 0;   // non-critical

    while (1) {
        int meas_rsrp   = get_meas_rsrp();
        int best_cell   = get_best_cell();
        int target_cell = get_target_cell();

        if (rrc_state == RRC_CONNECTED && meas_rsrp <
            bad_rsrp)
            poor_link_count++;
        else
            poor_link_count = 0;

        if (poor_link_count > N && mobility_flag) {
            if (target_cell == best_cell)
                ho_decision = 1;
            else
                ho_decision = 0;
        } else {
            ho_decision = 0;
        }

        debug_counter++;

        // Safety property : never handover to a non-best
            cell
        assert(!(ho_decision && (target_cell !=
            best_cell)));
    }
}
```

Fig. 1. Representative gNB handover loop used for CRV analysis.

Violation of ϕ indicates an incorrect handover triggered entirely by corrupted internal state, even when signalling messages appear valid.

In real deployments, safety properties often involve timing windows or temporal conditions (e.g., "handover shall not occur within 10 ms of the last measurement update"). Such properties are usually expressed informally in specifications. For clarity, we use the above atomic property, but the CRV principles extend naturally to more complex, time-dependent conditions.

C. CRVs and Non-CRVs for Safety Property ϕ

A variable is a CRV if altering only that variable can change the truth of the safety property. Formally, writing $\phi(s)$ for property ϕ evaluated on state s and $s[v \leftarrow x]$ for the state obtained by assigning x to variable v:

$$v \text{ is a CRV} \iff \exists x_1, x_2 : \ \phi(s[v \leftarrow x_1]) \neq \phi(s[v \leftarrow x_2]).$$

Applying this criterion to Fig. 1 yields:

- **CRVs:** `ho_decision`, `target_cell`, `best_cell`. Changing any of these can directly falsify ϕ.
- **Non-CRVs:** `rrc_state`, `poor_link_count`, `mobility_flag`, `debug_counter`. Although they affect control-flow structure, no pair of their values makes ϕ change its truth value. Hence they do not require protection for this safety condition.

Intuitively, even if an SEU changes `rrc_state`, `poor_link_count` or `mobility_flag`, the inner condition `target_cell == best_cell` still prevents a handover to a non–best cell. These variables can affect when the decision is taken, but they cannot by themselves force a violation of ϕ. Hence they do not need protection for this particular safety property.

D. Why Static Slicing Over-Approximates

Static program slicing [15] identifies variables that may influence a particular program point under any execution path. For the assertion in Fig. 1, slicing naturally retains all variables participating in the control path leading to `ho_decision`, including `rrc_state`, `poor_link_count` and `mobility_flag`. However, slicing answers a different question: *which variables syntactically influence the computation?* For SEU resilience, the relevant question is: *which variables can change the truth of the safety property when flipped?*. Variables such as `poor_link_count` or `mobility_flag` appear in the slice but cannot create a property violation under any single-bit corruption. Slicing therefore over-approximates the set of variables that matter for safety. CRV analysis resolves this by being property-directed: a variable is marked relevant only when its corruption can actually break ϕ.

E. How the CRV Method Fixes the Issue

The CRV method evaluates each variable directly against the safety property. It checks whether a single corruption of that variable can produce a counterexample to ϕ. This reframes the analysis from: "Does this variable participate in the control path?" to "Can this variable cause a safety violation if flipped?"

Using this criterion:

- Only `ho_decision`, `target_cell` and `best_cell` can falsify ϕ.
- All other variables may affect timing or performance but cannot cause a violation of the property.

This illustrates the core advantage of CRV-based reasoning: slicing includes too many variables, while CRV identification isolates exactly the minimal set that must be protected for SEU resilience. These CRVs become the precise targets for SEU injection and property checking in our architecture (Section IV).

IV. ARCHITECTURE FOR SEU-RESILIENT NETWORK CONTROL VIA CRV IDENTIFICATION

A. Design Scope

The architecture applies to control-plane, user-plane and management functions across the gNB and 5G Core, including mobility handling, handover control, QoS enforcement, slice behaviour, session continuity and UPF-level forwarding decisions. The method does not modify signalling procedures or

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on October 30,2026 at 08:38:53 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 CRV Identification for Network Control Functions

Require: Network module M, safety property ϕ
Ensure: Set of conditionally relevant variables C

$V \leftarrow \text{EXTRACTCANDIDATES}(M)$
$C \leftarrow \emptyset$
for each variable $v \in V$ **do**
  $M' \leftarrow \text{INSTRUMENTWITHSEU}(M, v)$
  result $\leftarrow \text{MODELCHECK}(M', \phi)$
  if result indicates violation caused by v **then**
    $C \leftarrow C \cup \{v\}$
  end if
end for
return C

protocol formats and is fully compatible with 3GPP and O-RAN software.

B. Overview of the CRV Identification Workflow

CRV identification is performed through a structured workflow that builds directly on the motivating example in Section III. The key idea is to test whether a single bit-flip in each variable can change the truth of the safety property. If it can, the variable is a CRV; otherwise it is not. This workflow produces the set of variables that must be protected. The remainder of this section explains how each step is carried out.

C. CRV Identification Flow

The flow proceeds as follows:

1) **Specify the safety property** (e.g., "handover must not target a non-best cell").
2) **Extract candidate variables** using static slicing [15] to obtain all variables that may influence the decision.
3) **Inject a single SEU** into each candidate and check if the safety property ϕ can be violated.
4) **Mark the variable as a CRV** only if some SEU causes ϕ to become false.

Variables that never falsify ϕ under any single-bit upset are safely ignored. We use the CBMC bounded model checker [17] to evaluate each instrumented program and determine whether the injected SEU can violate the property ϕ.

D. CRV-Identification Algorithm

We now describe how the CRV-identification algorithm operates and how each step contributes to determining whether a variable is safety-critical. The function $\text{EXTRACTCANDIDATES}(M)$ performs slicing and dependence analysis, returning a superset V of variables that may influence ϕ. Each variable is then tested under a one-time SEU injection to determine whether it is safety-critical.

```c
/* Global flag to ensure a single SEU in one execution */
static int seu_fired = 0;

void inject_seu_once(int *v) {
    if (!seu_fired) {
        int bit = nondet_int();
        __CPROVER_assume(bit >= 0 &&
                         bit < (int)(sizeof(*v) * 8));
        int mask = 1 << bit;
        *v = *v ^ mask;      /* flip the chosen bit */
        seu_fired = 1;
    }
}
```

Fig. 2. Helper function for injecting a one-time SEU bit-flip.

```c
/* Before instrumentation */
x = compute_metric(...);
use_decision(x);

/* After instrumentation for CRV testing */
x = compute_metric(...);
inject_seu_once(&x);   // one-time bit-flip
use_decision(x);
```

Fig. 3. SEU instrumentation inserted before variable use.

E. Single-Event Upset (SEU) Model

The algorithm relies on a standard single-event upset model: at most one transient bit-flip may occur per execution, affecting only one integer variable. Fig. 2 shows the helper routine used throughout the analysis.

F. Instrumenting a Variable for SEU Evaluation

To test whether a variable v is a CRV, every use of v is instrumented with the one-time SEU injection shown in Fig. 2. Fig. 3 illustrates the transformation applied by $\text{INSTRUMENTWITHSEU}(M, v)$. This transformation links the algorithmic step $\text{INSTRUMENTWITHSEU}(M, v)$ with the actual SEU injection function in Fig. 2. The model checker then determines whether the injected fault can break the safety property ϕ.

G. Applying CRV-Guided Protection in RAN Control Paths

CRV-guided protection applies naturally to RAN control loops such as scheduling, HARQ handling and RRC state transitions. Protecting every variable is impractical, whereas CRV-guided protection limits safeguards to the few variables that actually matter for safety. This keeps the control path stable under SEU conditions while retaining low overhead and requiring no changes to the radio stack.

CRV Set Size and Protection Scope. CRV relevance is property- and state-dependent. In practice, the union of CRVs across reachable states is typically much smaller than the full program state, since only variables that can directly influence the safety outcome are included. The method ensures that variables capable of causing safety violations under any reachable condition are protected, without assuming that all CRVs are simultaneously active.

Bit flips in non-CRV variables may still affect intermediate computations or system behaviour. However, CRV-guided

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on October 30,2026 at 08:38:53 UTC from IEEE Xplore. Restrictions apply.

Without CRV Awareness	With CRV-Guided Protection
Large set of variables treated as critical	Only safety-affecting variables treated as critical
High cost (full duplication / ECC)	Low cost (protect CRVs only)
SEU effects difficult to trace	SEU influence bounded and predictable
Overhead grows with code size	**Overhead grows only with number of CRVs**

TABLE I
EFFECT OF APPLYING CRV-GUIDED SELECTIVE PROTECTION.

protection focuses specifically on preventing safety property violations. In practice, this selective protection can be combined with existing mechanisms such as periodic state refresh, runtime monitoring, or system-level fault detection to provide broader robustness. The protection overhead therefore scales primarily with the number of CRVs rather than the total number of program variables.

V. Discussion and Deployment Considerations

CRV-guided protection integrates directly into existing 5G/6G software at the C source level. Once the CRVs are identified, small code-level checks—such as range checks or duplicate-and-compare updates—can be added only around these variables, without affecting protocol handling or 3GPP procedures.

A. Software-Only Deployment

Most RAN and Core control logic is implemented in software, which makes CRV identification easy to integrate into current workflows. The approach requires no kernel, driver or hardware changes and can be adopted gradually within existing testing pipelines.

B. Use in Disaggregated and O-RAN Systems

In O-RAN architectures, xApps and rApps expose explicit control logic. CRVs can be identified directly within these applications, allowing selective protection with low runtime overhead—important for near-real-time loops.

C. Edge and Data-Center Settings

Core functions deployed in edge sites or data centres often operate under high load and temperature. Protecting only CRVs limits the spread of SEU-induced corruption even on servers without full ECC support.

D. Summary

The method avoids full-state hardening and safeguards only the variables that can affect the safety outcome. This keeps the solution lightweight and suitable for both current 5G deployments and future 6G systems.

Example	T	S	M	η (%)
UPF Selection	12	8	3	37%
Handover Trigger	11	7	2	28%
QoS Slice Classification	10	7	2	28%
Mobility State Continuity	13	9	3	33%

TABLE II
CRV-GUIDED ANALYSIS FOR FOUR REPRESENTATIVE NETWORK PROGRAMS.

VI. Case Study and Experimental Results

To understand how CRVs arise in realistic network software, we manually inspected selected modules from the OpenAirInterface (OAI) 5G RAN and Core codebase [16]. OAI contains realistic 5G control paths with many internal state variables. This reflects the CRV principle: among many variables in a control path, only a subset can affect a safety property under a fault.

Representative programs. Motivated by the patterns seen in the OAI code, we wrote four small network–oriented programs that capture typical 5G decision logic. These programs are not literal extracts; instead, they model common behaviours such as threshold checks, state transitions, policy lookups and QoS classification. Each program is paired with a safety property ϕ, and CRVs are identified by combining static slicing with guided SEU injection. This setup gives us controlled, repeatable experiments while keeping the examples close to real network behaviour.

A. Efficiency of CRV Detection

To quantify the benefit of CRV analysis, we use the following quantities:

- T: total variables examined,
- S: variables identified as relevant by static slicing,
- M: variables eliminated by CRV analysis as non-critical,
- η: efficiency gain over slicing.

$$\eta = \left(\frac{M}{S}\right) \times 100\%.$$

B. Evaluation on Four Network Examples

Across all four programs, static slicing marks several variables as relevant, but CRV analysis retains only those that can actually falsify the safety property ϕ. Between 28% and 37% of the sliced variables are eliminated as non–critical (column η in Table II). This shows how a property–directed analysis can be more precise than purely syntactic slicing when the goal is SEU resilience. Each example models a common 5G control pattern; *UPF Selection* represents slice-aware forwarding decisions in the Core; *Handover Trigger* captures mobility-driven RSRP threshold logic in the RAN; *QoS Slice Classification* reflects policy-based QoS marking and slice mapping; and *Mobility State Continuity* models RRC state transitions driven by timers and counters. These abstractions mirror the structure of the OAI routines examined earlier and allow controlled evaluation of CRV behaviour.

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on October 30,2026 at 08:38:53 UTC from IEEE Xplore. Restrictions apply.

C. Summary

These findings echo the results from the OAI inspection: only a small fraction of variables are conditionally relevant, while static slicing alone includes many unnecessary variables. CRV analysis filters out such variables and retains only those capable of changing the truth of the safety property. This selective, property-driven view offers a practical path to SEU resilience in RAN and Core software.

While our experiments focus on four representative network examples guided by OAI behaviour, earlier CRV work has demonstrated scalability on large embedded-controller suites. The same automation pipeline can be extended to full RAN and Core modules in future work as our tooling matures.

Practical Scalability and Automation. While the current evaluation uses four representative programs derived from patterns observed in the OAI codebase, the CRV identification workflow is designed to be automatable once safety properties and instrumentation hooks are defined. Static slicing, instrumentation, and CBMC-based verification can be executed in batch across modules, making the approach practical for large codebases.

In practice, CRV identification is performed offline during development, testing, or certification phases rather than during runtime. The computational cost is therefore comparable to existing formal verification and large-scale testing workflows already used in telecom software validation. Importantly, CRV analysis is property-scoped rather than system-scoped, meaning only variables influencing a given safety property are analysed at a time. Safety properties themselves are typically already defined through protocol specifications, conformance tests, or runtime monitoring rules, and CRV analysis is intended to complement these existing workflows.

VII. Conclusion and Future Work

This paper applied conditionally relevant variables (CRVs) to improve SEU resilience in 5G/6G network control software. Instead of hardening the full program state or duplicating control paths, we identify only those variables whose corruption can change the truth of a safety property. These variables form the CRV set and represent the natural targets for protection.

We combined static slicing, a simple SEU fault model, and CBMC-based model checking to identify CRVs in C implementations of network control routines. The motivating handover example showed how CRVs differ from variables that influence only timing or performance. The OAI-inspired case study showed that only a subset of sliced variables are conditionally relevant for typical network safety properties.

The approach fits existing RAN and Core software because it operates directly at the C source level and does not require changes to 3GPP procedures or protocol formats. Once CRVs are identified, lightweight safeguards such as range checks, simple redundancy, or periodic refresh can be applied only to these variables, limiting protection overhead.

Future work will focus on scaling CRV analysis to larger network modules, including full RRC and UPF implementations, with increased automation of property handling and candidate extraction. We also plan to evaluate CRV-guided protection under realistic traffic and mobility patterns, and extend analysis across CU-CP, CU-UP, and Core boundaries. In the longer term, we aim to develop tooling and design guidelines to make CRV analysis a routine step in 5G/6G software development.

While this work focuses on single-bit SEU faults, the CRV concept is independent of the specific fault model. The same workflow can be extended to multi-bit upsets, word-level corruption, and other transient fault classes such as timing faults or transient computation errors, provided these faults can be expressed as software-visible state perturbations.

References

[1] Cisco Systems, “Cosmic radiation may have triggered router malfunction,” *NetworkWorld*, Sep. 2016. [Online]. Available: https://tinyurl.com/cisco-seu-2016

[2] Juniper Networks, ”Understanding the Soft Error Recovery Feature on Packet Forwarding Engine (PFE),” Technical Article, Jul. 2019. [Online]. Available: https://supportportal.juniper.net/s/article/EX-Understanding-the-soft-error-recovery-feature-on-PFE

[3] JEDEC Solid State Technology Association, ”Measurement and Reporting of Alpha Particle and Terrestrial Cosmic Ray–Induced Soft Errors in Semiconductor Devices,” Standard JESD89B, 2012. [Online]. Available: https://www.jedec.org/standards-documents/docs/jesd-89a

[4] B. Schroeder and G. A. Gibson, ”A large-scale study of failures in high-performance computing systems,” in *Proc. Int. Conf. Dependable Systems and Networks*, 2009. [Online]. Available: https://research.google/pubs/dram-errors-in-the-wild-a-large-scale-field-study/

[5] Texas Instruments, ”Clock Jitter Cleaners – Total Ionizing Dose and Single-Event Effects Test Results,” Radiation Test Summary for LMK-series devices. [Online]. Available: https://www.ti.com/product-category/clocks-timing/clock-jitter-cleaners/overview.html

[6] Anonymous, ”Safeguarding Controller Software from Single Event Upset via Conditional Variable Verification”, APLAS SRC, 2025.

[7] Anonymous, ”A Formal Verification Approach to Safeguard Controller Variables from Single Event Upset”, arXiv preprint, 2025.

[8] AMD (Xilinx), ”Soft Error Mitigation (SEM) Core,” Product Guide PG187. [Online]. Available: https://www.amd.com/en/products/adaptive-socs-and-fpgas/intellectual-property/sem.html

[9] Lattice Semiconductor, ”Single Event Upset (SEU) Report for Nexus Platform,” FPGA-TN-02174. [Online]. Available: https://www.latticesemi.com/Products/FPGAandCPLD/CertusPro-NX

[10] M. Berg, K. LaBel, and J. Pellish, ”Kintex UltraScale Field Programmable Gate Array Single Event Effects Characterization Summary,” NASA Technical Report, 2018. [Online]. Available: https://ntrs.nasa.gov/api/citations/20180004740/downloads/20180004740.pdf

[11] International Telecommunication Union (ITU-T), ”Protection Against Interference,” K-Series Recommendations. [Online]. Available: https://www.itu.int/rec/T-REC-K/en

[12] Australian Bureau of Meteorology, Space Weather Services, ”Satellite Communications and Space Weather.” [Online]. Available: https://www.sws.bom.gov.au/Educational/1/3/2

[13] K. Bedingfield, R. B. Leach, and M. B. Alexander, ”Spacecraft System Failures and Anomalies Attributed to the Space Environment,” NASA Reference Publication 1390, 1996. [Online]. Available: https://ntrs.nasa.gov/api/citations/19960050463/downloads/19960050463.pdf

[14] R. Baumann, ”Radiation-induced soft errors in advanced semiconductor technologies,” *IEEE Trans. on Device and Materials Reliability*, vol. 5, no. 3, pp. 305–316, 2005.

[15] M. Weiser, “Program slicing,” *IEEE Trans. Softw. Eng.*, vol. SE-10, no. 4, pp. 352–357, July 1984.

[16] OpenAirInterface, ”The OpenAirInterface 5G Project”, [Online]. Available: https://openairinterface.org/

[17] E. Clarke, D. Kroening, and F. Lerda, “CBMC: A bounded model checker for C and C++ programs,” in *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, pp. 168–176, Springer, 2004.

Authorized licensed use limited to: International Institute of Information Technology Bangalore. Downloaded on October 30,2026 at 08:38:53 UTC from IEEE Xplore. Restrictions apply.